

Analysis

August 5, 2025

1 Introduction

The goal of this project is to analyze data provided by the Government of Alberta. The dataset contains measurements of various species of fish found in Alberta's waterbodies and the concentration of mercury (Hg) in their tissue, measured by mg/kg.

An explanation of all the data columns can be found in the `hg-in-fish-column-descriptions.xlsx` file in the data folder. The `hg-in-fish.xlsx` file contains the data used for analysis. The source of the datasets can be found here: <https://open.alberta.ca/opendata/chemical-monitoring-in-local-foods-mercury-in-fish>.

1.1 Exploring the Data

The first step is to get a sense of the data.

```
[1]: import pandas as pd
import numpy as np
import plotly.express as px
import statsmodels.api as sm
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.stats.multicomp as multi
from statsmodels.stats.multicomp import pairwise_tukeyhsd
from statsmodels.formula.api import ols

df = pd.read_excel('data/hg-in-fish.xlsx')
```

```
[2]: # Checking the number of columns and rows in the dataset
df.shape
```

```
[2]: (6373, 25)
```

There are 25 columns and 6373 rows.

```
[3]: # Showing the first 5 rows of the dataset
df.head()
```

```
[3]:   Program_ID  Sample_ID Collection Date Date_flag  Waterbody Name \
0  21-FM-LKWH-01  21-FM-LKWH-01      2021-09-28         0  40 Mile Reservoir
```

1	21-FM-LKWH-02	21-FM-LKWH-02	2021-09-28	0	40 Mile Reservoir
2	21-FM-LKWH-03	21-FM-LKWH-03	2021-09-28	0	40 Mile Reservoir
3	21-FM-LKWH-04	21-FM-LKWH-04	2021-09-28	0	40 Mile Reservoir
4	21-FM-LKWH-05	21-FM-LKWH-05	2021-09-28	0	40 Mile Reservoir

	Waterbody Type	Latitude	Longitude	Land Use Region	FWMIS ID	...	\
0	Reservoir	49.64363	-111.393672	South Saskatchewan	6813	...	
1	Reservoir	49.64363	-111.393672	South Saskatchewan	6813	...	
2	Reservoir	49.64363	-111.393672	South Saskatchewan	6813	...	
3	Reservoir	49.64363	-111.393672	South Saskatchewan	6813	...	
4	Reservoir	49.64363	-111.393672	South Saskatchewan	6813	...	

	Fork LengthMeasurement	Total Length (mm)	Total LengthMeasurement	\
0	Field	439	Field	
1	Field	425	Field	
2	Field	418	Field	
3	Field	409	Field	
4	Field	398	Field	

	LengthEstimateFactor	Weight (g)	WeightMeasurement	Maturity	Age (years)	\
0	NaN	610	Field	Mature	Unknown	
1	NaN	647	Field	Mature	9	
2	NaN	710	Field	Mature	6	
3	NaN	591	Field	Mature	Unknown	
4	NaN	600	Field	Mature	9	

	Aging Structure	Hg (mg/kg)
0	Otolith	0.136
1	Otolith	0.104
2	Otolith	0.057
3	Otolith	0.082
4	Otolith	0.143

[5 rows x 25 columns]

```
[4]: # Listing all the columns and their attributes
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6373 entries, 0 to 6372
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Program_ID            6373 non-null   object
1   Sample_ID             6371 non-null   object
2   Collection Date       6373 non-null   datetime64[ns]
3   Date_flag             6372 non-null   object
4   Waterbody Name        6373 non-null   object
```

```

5   Waterbody Type      6373 non-null  object
6   Latitude           6373 non-null  object
7   Longitude          6373 non-null  object
8   Land Use Region    6373 non-null  object
9   FWMIS ID           6373 non-null  object
10  Species Code       6373 non-null  object
11  Common Name        6373 non-null  object
12  Binomial Name      6373 non-null  object
13  Sex                6373 non-null  object
14  Fork Length (mm)    6373 non-null  object
15  Fork LengthMeasurement 5291 non-null  object
16  Total Length (mm)  6373 non-null  object
17  Total LengthMeasurement 5174 non-null  object
18  LengthEstimateFactor 2 non-null    object
19  Weight (g)         6373 non-null  object
20  WeightMeasurement  6236 non-null  object
21  Maturity           6373 non-null  object
22  Age (years)        4787 non-null  object
23  Aging Structure    4787 non-null  object
24  Hg (mg/kg)         6373 non-null  object
dtypes: datetime64[ns](1), object(24)
memory usage: 1.2+ MB

```

Reviewing the `hg-fish-column-descriptions.xlsx` file shows the most relevant columns for this analysis seem to be: - Waterbody Name - Waterbody Type - Common Name - Sex - Fork Length (mm) - Total Length (mm) - Weight (g) - Maturity - Age (years) - Hg (mg/kg)

1.2 Preparing the Data for Analysis

The data need to be cleaned and formatted before analysis can be done. There are many unnecessary columns that can be dropped from the dataframe and numeric columns that are assigned the wrong datatype.

```

[5]: # Defining the columns to keep in the updated dataframe
cols = ['Waterbody Name', 'Waterbody Type', 'Common Name', 'Sex', 'Fork Length_
↪(mm)', 'Total Length (mm)', 'Weight (g)', 'Maturity', 'Age (years)', 'Hg (mg/
↪kg)']

```

```

[6]: # Updating the dataframe
df = df[cols]

```

```

[7]: # Confirming the dataframe has been updated
df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6373 entries, 0 to 6372
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -

```

```

0   Waterbody Name      6373 non-null   object
1   Waterbody Type      6373 non-null   object
2   Common Name         6373 non-null   object
3   Sex                 6373 non-null   object
4   Fork Length (mm)    6373 non-null   object
5   Total Length (mm)   6373 non-null   object
6   Weight (g)          6373 non-null   object
7   Maturity            6373 non-null   object
8   Age (years)         4787 non-null   object
9   Hg (mg/kg)          6373 non-null   object
dtypes: object(10)
memory usage: 498.0+ KB

```

After updating the dataframe to contain only the relevant data for analysis, `df.info()` shows that there are columns in the dataset which data types should be *numeric* and not *object*. In order to perform the analysis, the data type of these columns must be changed to numeric. After transforming the columns any non-number data will be changed to NaN. The rows containing those NaN values must be removed from the dataframe.

```

[8]: # Assigning the columns needed to converted into numeric values into a variable
columns_to_convert = ['Fork Length (mm)', 'Total Length (mm)', 'Weight (g)',
    ↪ 'Age (years)', 'Hg (mg/kg)']

# Converting the columns to numeric data type
df[columns_to_convert] = df[columns_to_convert].apply(lambda x: pd.
    ↪ to_numeric(x, errors='coerce'))

```

Now that the data type of the columns has been changed, the number of NaN values need to be identified. For now, only the NaN values in the Hg (mg/kg) column need to be dropped, as that is the dependent variable being investigated. These values can be identified using `df.isnull().sum()`.

```

[9]: # Calculating the number of NaN values in the Hg (mg/kg) column
print('The number of null values in the Hg (mg/kg) column is', df['Hg (mg/kg)'].
    ↪ isnull().sum())

```

The number of null values in the Hg (mg/kg) column is 6

```

[10]: # Dropping the NaN values
df.dropna(subset=['Hg (mg/kg)'], inplace=True)

```

```

[11]: # Confirming that NaN values have been dropped
print('The number of null values in the Hg (mg/kg) column is', df['Hg (mg/kg)'].
    ↪ isnull().sum())

```

The number of null values in the Hg (mg/kg) column is 0

Now that all of the NaN values in the Hg (mg/kg) column have been removed, it will be helpful to visualize the distribution of the values in Hg (mg/kg). Using `plotly.express` makes this very easy to do.

```
[12]: # Checking to see the distribution of values in Hg (mg/kg)
hg_box = px.box(df['Hg (mg/kg)'])
hg_box.show()
```

There appear to be many outliers in the data. A function can be built that will identify and count outliers in the Hg (mg/kg) column.

```
[13]: # Function to identify the number of outliers
def get_number_of_outliers_iqr(data):
    Q1 = data.quantile(0.25)
    Q3 = data.quantile(0.75)
    IQR = Q3 - Q1
    outliers = ((data < (Q1 - 1.5 * IQR)) | (data > (Q3 + 1.5 * IQR))) #
    ↪ Outlier detection rule
    number_of_outliers = sum(outliers) # Count the outliers
    return number_of_outliers
```

```
[14]: # Calculating the number and percent of outliers
total_data_points = len(df['Hg (mg/kg)'])
number_of_outliers_hg = get_number_of_outliers_iqr(df['Hg (mg/kg)'])
percentage_of_outliers = (number_of_outliers_hg / total_data_points) * 100
print(f'Number of outliers: {number_of_outliers_hg}')
print(f'Percentage of outliers: {percentage_of_outliers:.1f}%')
```

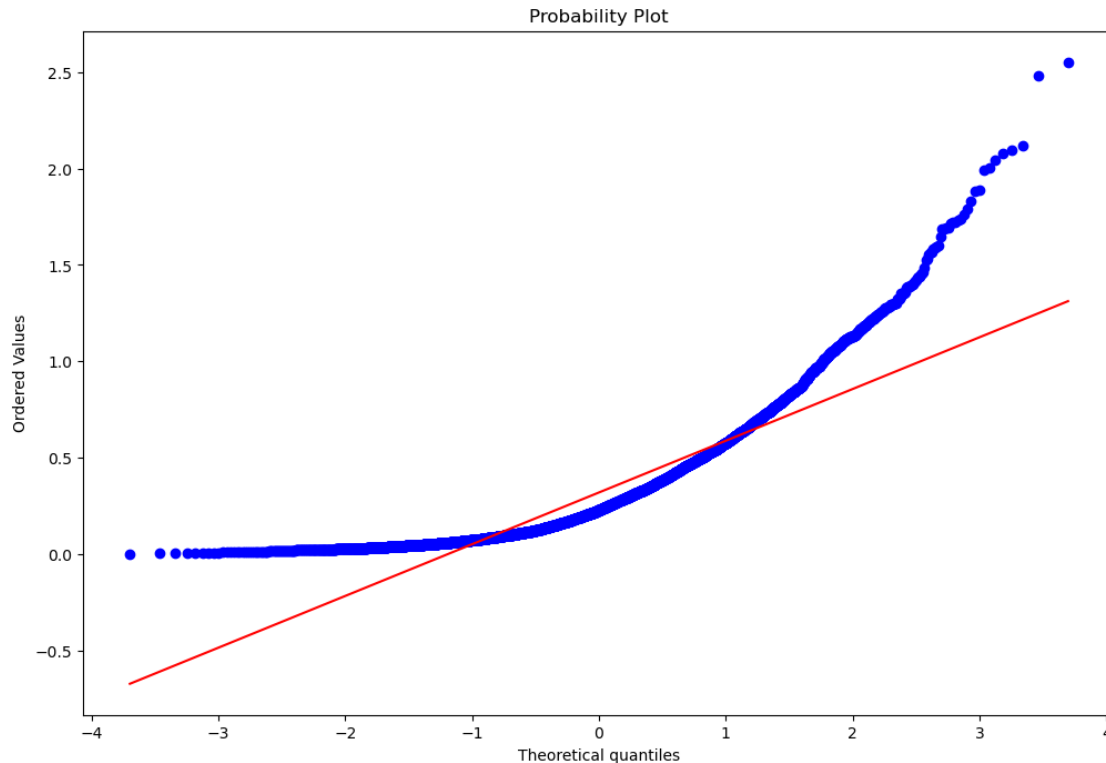
Number of outliers: 263
Percentage of outliers: 4.1%

There are 263 outliers in the original Hg (mg/kg) column, which is 4.1% of the total rows.

```
[15]: hg_hist = px.histogram(df['Hg (mg/kg)'], nbins=50)
hg_hist.show()
```

The boxplot and histogram show that the data contain many outliers that skew the data. This suggests that the data are not normally distributed, but this can be confirmed using `matplotlib` and `scipy` to create a Q-Q probability plot. If the data are normally distributed, the data should follow or “hug” the plotted line in a linear fashion.

```
[16]: # Creating a Q-Q probability plot to check for normality
plt.figure(figsize=(12, 8))
stats.probplot(df['Hg (mg/kg)'], plot=plt)
plt.show()
```



The Q-Q probability plot shows the data are not normally distributed. Performing a **log10 transformation** could help normalize the data and remove outliers, which can be done using **numpy**.

```
[17]: # Performing a log10 transformation using numpy and adding those values to a
      ↪ new column called log10_hg
df['log10_hg'] = np.log10(df['Hg (mg/kg)'])
```

After the transformation the outliers can be recalculated.

```
[18]: # Calculating the number and percent of outliers after the log10 transformation
total_data_points = len(df['log10_hg'])
number_of_outliers = get_number_of_outliers_iqr(df['log10_hg'])
percentage_of_outliers = (number_of_outliers / total_data_points) * 100
print(f'Number of outliers: {number_of_outliers}')
print(f'Percentage of outliers: {percentage_of_outliers:.1f}%')
```

Number of outliers: 13

Percentage of outliers: 0.2%

After the transformation, the number of outliers is now 13, which is 0.2% of the rows.

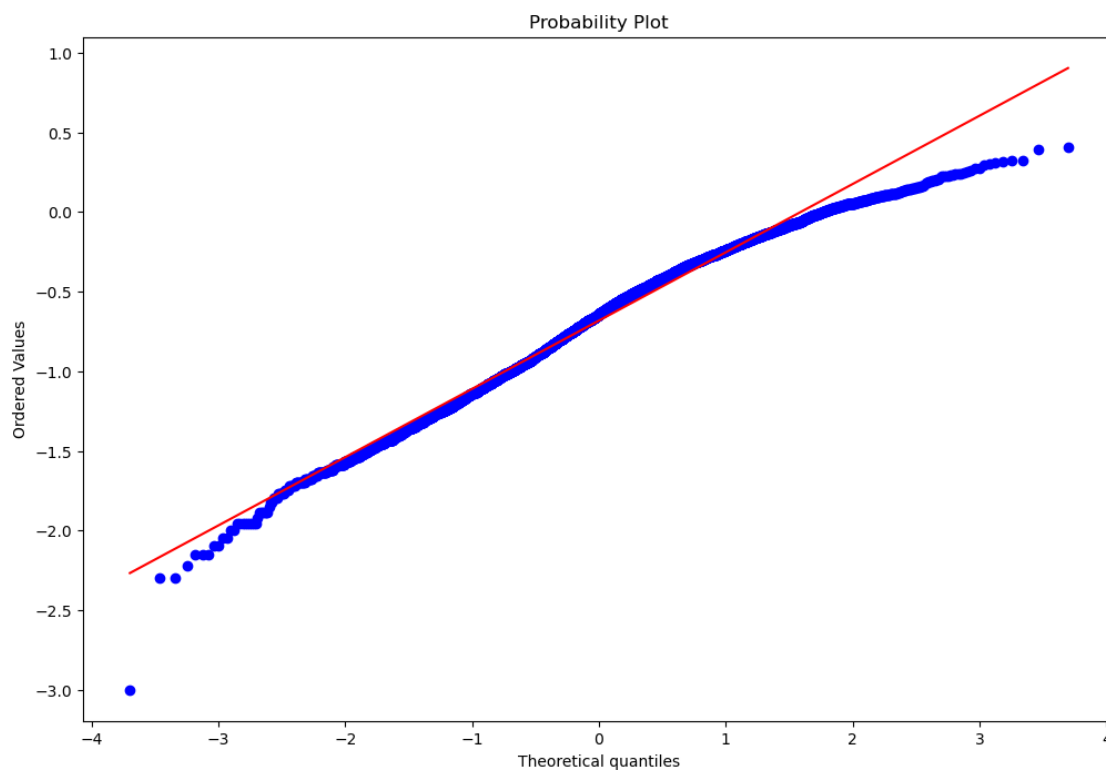
Now that the Hg (mg/kg) values have been transformed and added as a new column, it's a good idea to check if any new NaN values were generated as a result of the transformation. If any NaN values are present, the statistical tests will not work.

```
[19]: # Checking for any NaN values
print('The number of null values in the log10_hg column is', df['log10_hg'].
      ↪isnull().sum())
```

The number of null values in the log10_hg column is 0

Now a new Q-Q probability plot can be created to see if the transformation helped to normalize the Hg (mg/kg) data.

```
[20]: # Creating a new Q-Q probability plot on the transformed data
plt.figure(figsize=(12, 8))
stats.probplot(df['log10_hg'], plot=plt)
plt.show()
```



After the transformation the data appear to be approximately normal, which indicates parametric statistical tests can be used for analysis. The data can also be replotted to see the difference the transformation made.

```
[21]: log10_box = px.box(df['log10_hg'])
log10_box.show()
```

```
[22]: log10_hist = px.histogram(df['log10_hg'], nbins=50)
log10_hist.show()
```

The data are much less skewed and most of the outliers have been removed. There still are a few, 0.2% of the data, but likely not enough to substantially affect the analysis.

1.2.1 Maturity Analysis

The first independent variable that will be investigated is the `Maturity` variable.

Null hypothesis: there are no significant differences between the means of Hg (mg/kg) in each `Maturity` group.

Hypothesis: there is a significant difference between the means of Hg (mg/kg) in each `Maturity` group. Mature fish will have a significantly larger concentration of Hg (mg/kg).

In order to investigate the data properly, a new dataframe will be generated that only contains the `Maturity`, `log10_hg`, and `Hg (mg/kg)` columns. In addition, the new dataframe is created so that `dropna()` can be used as there might be rows in the `Maturity` column that have NaN values. Those will need to be removed. This was not done on the entire dataset earlier as it might unnecessarily remove rows that can be used for this analysis but not for another.

```
[23]: # Creating a new dataframe that only contains the relevant data for the
      ↪Maturity analysis and dropping NaN values
df_mat = df[['Maturity', 'log10_hg', 'Hg (mg/kg)']].dropna()
```

Now that the new dataframe has been created and any NaN values have been removed, the groups (the unique values) can be identified using `value_counts()`.

```
[24]: df_mat['Maturity'].value_counts()
```

```
[24]: Maturity
Mature      3527
Unknown     2590
Immature     232
Triploid      18
Name: count, dtype: int64
```

It appears that there is a group titled *Unknown*, which won't be useful for the analysis, so they will be removed from the dataframe.

```
[25]: # Removing the rows that contain 'Unknown' values
df_mat = df_mat[~df_mat.isin(['Unknown']).any(axis=1)]
```

To test if there is a significant difference between the means of Hg (mg/kg) in the different stages of maturity, a one-way ANOVA can be used. First, the data must be separated into groups before the test can be performed. Afterwards, the ANOVA can be performed.

```
[26]: # Separating the data into groups
maturity_levels = ['Mature', 'Immature', 'Triploid']

groups = {}
for maturity in maturity_levels:
```

```

    groups[maturity] = df_mat[df_mat['Maturity'] == maturity]['log10_hg'].
    ↪dropna()

# Number of groups
num_groups = len(groups)

# The total number of observations
num_observations = sum([len(group) for group in groups.values()])

# Computing degrees of freedom
df_between_groups = num_groups - 1
df_within_groups = num_observations - num_groups
print(f'Between-groups Degrees of Freedom: {df_between_groups}')
print(f'Within-groups Degrees of Freedom: {df_within_groups}')

# Performing the one-way ANOVA using values from the dictionary
F_statistic, p_value = stats.f_oneway(*groups.values())
print(f'The F-statistic is: {F_statistic:.3f}')
print(f'The p-value is: {p_value:.3f}')

```

```

Between-groups Degrees of Freedom: 2
Within-groups Degrees of Freedom: 3774
The F-statistic is: 41.401
The p-value is: 0.000

```

The ANOVA revealed that there was a statistically significant difference in Hg (mg/kg) concentration between at least two groups, $F(2, 3774) = 41.401$, $p < 0.001$. A Tukey HSD test can be used to identify which variables have a significant difference between their means.

The Tukey HSD test requires that all the groups must be combined into a single dataframe.

```

[27]: # Create a new DataFrame with structure suitable for the Tukey HSD test
dataframes = []
groups = ['Mature', 'Immature', 'Triploid']
for group in groups:
    df_mat_group = df_mat[df_mat['Maturity'] == group][['log10_hg']].copy()
    df_mat_group['group'] = group
    dataframes.append(df_mat_group)
tukey_df_type = pd.concat(dataframes)

# Drop NaN values to prevent errors
tukey_df_type = tukey_df_type.dropna()

# Apply the Tukey HSD test
results = pairwise_tukeyhsd(endog=tukey_df_type['log10_hg'],
    ↪groups=tukey_df_type['group'], alpha=0.05)
print(results)

```

Multiple Comparison of Means - Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
Immature	Mature	0.1975	0.0	0.13	0.2651	True
Immature	Triploid	-0.4152	0.0002	-0.659	-0.1715	True
Mature	Triploid	-0.6128	0.0	-0.8482	-0.3773	True

Tukey's HSD Test for multiple comparisons found that the mean value of Hg (mg/kg) was significantly different between mature and immature ($p < 0.001$), immature and triploid ($p < 0.001$), and mature and triploid ($p < 0.001$).

```
[28]: # Calculate the mean 'Hg (mg/kg)' value for each maturity group and sort them
      ↪from largest to smallest
mean_values = df_mat.groupby('Maturity')['Hg (mg/kg)'].mean().round(3).
      ↪sort_values(ascending=False)
print(mean_values)
```

```
Maturity
Mature      0.369
Immature    0.228
Triploid     0.062
Name: Hg (mg/kg), dtype: float64
```

Mature fish had the highest concentration of Hg (mg/kg) at 0.369 mg/kg, followed by immature fish at 0.228 mg/kg, and lastly triploid fish at 0.062 mg/kg.

The values and their groups can be plotted with a boxplot to visualize the distribution.

```
[29]: # Plotting a boxplot to visualize the distribution of data for all groups
box = px.box(df_mat, x='Maturity', y='log10_hg')
box.show()
```

1.2.2 Weight Analysis

Null hypothesis: there is no significant relationship between Weight (g) and Hg (mg/kg). Weight (g) will not significantly predict Hg (mg/kg).

Hypothesis: Weight (g) will significantly predict Hg (mg/kg). As Weight (g) increases, the concentration of Hg (mg/kg) will significantly increase.

As before, a different dataframe is created to contain only Weight (g) and log10_hg columns. Any possible NaN values are also dropped.

```
[30]: # Creating a new dataframe with only the data needed for the weight analysis
      ↪and dropping any NaN values
df_weight = df[['Weight (g)', 'log10_hg']].dropna()
```

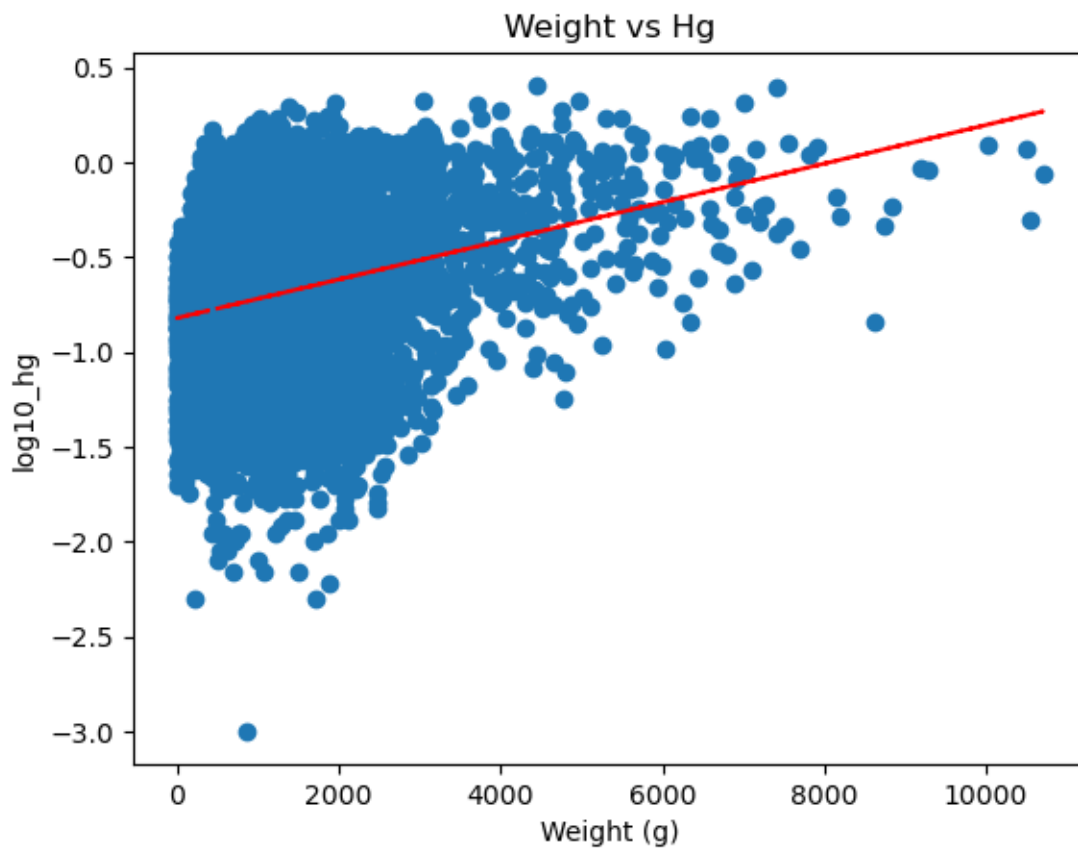
As both the Weight (g) and Hg (mg/kg) variables are numeric, the analysis method will be different. First, using matplotlib, a scatter plot with a trend line will be made to see if a potential linear relationship between both variables exists.

```
[31]: # Creating a scatter plot with a trend line to investigate and visualize a
      ↪ potential linear relationship between Weight (g) and Hg (mg/kg)
x = df_weight['Weight (g)']
y = df_weight['log10_hg']

plt.scatter(x, y)

z = np.polyfit(x, y, deg=1)
p = np.poly1d(z)

plt.plot(x, p(x), 'r--')
plt.title('Weight vs Hg')
plt.xlabel('Weight (g)')
plt.ylabel('log10_hg')
plt.show()
```



There appears to be a linear relationship between Weight (g) and Hg (mg/kg). A Pearson correlation can be computed to quantify the relationship between variables.

```
[32]: # Performing a Pearson correlation
corr, p_value = stats.pearsonr(df_weight['Weight (g)'], df_weight['log10_hg'])
df_degree = len(df_weight) - 2
print(f'The degrees of freedom is: {df_degree}')
print(f'The Pearsons correlation is: {corr:.3f}')
print(f'The p-value is: {p_value:.3f}')
```

The degrees of freedom is: 6228
The Pearsons correlation is: 0.262
The p-value is: 0.000

There was a positive correlation between the two variables, $r(6228) = 0.262$, $p < 0.001$. Performing a simple linear regression could provide more information about the relationship between these variables. This can be done using `statsmodels.api`. The independent variable is `Weight (g)` and the dependent variable is `Hg (mg/kg)`.

```
[33]: # Performing a simple linear regression
y = df_weight['log10_hg']
x = df_weight['Weight (g)']
x = sm.add_constant(x)
model = sm.OLS(y, x).fit()
model.summary()
```

[33] :

Dep. Variable:	log10_hg	R-squared:	0.069
Model:	OLS	Adj. R-squared:	0.069
Method:	Least Squares	F-statistic:	459.3
Date:	Tue, 05 Aug 2025	Prob (F-statistic):	2.32e-98
Time:	08:56:50	Log-Likelihood:	-3389.1
No. Observations:	6230	AIC:	6782.
Df Residuals:	6228	BIC:	6796.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	-0.8213	0.008	-97.004	0.000	-0.838	-0.805
Weight (g)	0.0001	4.75e-06	21.430	0.000	9.25e-05	0.000

Omnibus:	158.068	Durbin-Watson:	0.418
Prob(Omnibus):	0.000	Jarque-Bera (JB):	166.641
Skew:	-0.390	Prob(JB):	6.52e-37
Kurtosis:	2.813	Cond. No.	2.86e+03

Notes:

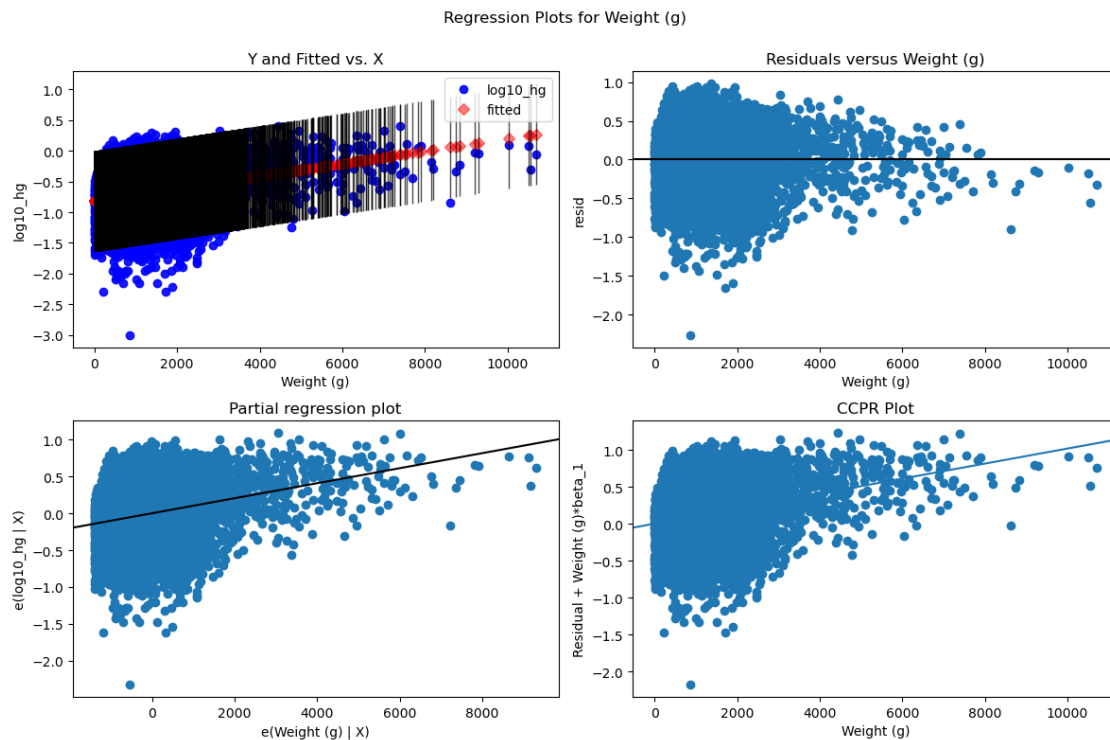
- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 2.86e+03. This might indicate that there are strong multicollinearity or other numerical problems.

The results from the linear regression are using the `log10` transformed `Hg (mg/kg)` data, held in the `log10_hg` column. Reversing the transformation on the constant calculated by the regression model will give the constant needed to build the linear equation that will quantify the relationship ($y = mx + b$). The reversal can be done with `numpy`.

```
[34]: # Reversing the transformation of the constant for the linear regression model
      ↪ using numpy
      b = np.exp(-0.8213)
      print(f'The untransformed constant for the linear regression is: {b:.3f}')
```

The untransformed constant for the linear regression is: 0.440

```
[35]: # Creating residual plots to check linear regression assumptions
      fig = plt.figure(figsize=(12, 8))
      fig = sm.graphics.plot_regress_exog(model, 'Weight (g)', fig=fig)
```



A simple linear regression was used to test if **Weight (g)** significantly predicted **Hg (mg/kg)**. The fitted regression model was: **Hg (mg/kg) = 0.0001 x Weight (g) + 0.440**. The overall regression was statistically significant; $R^2 = 0.069$, $F(1, 6228) = 459.3$, $p < 0.001$. The equation predicts a 0.0001 mg increase of Hg per one gram of weight gained. A one kilogram increase in weight would result in a 0.1mg increase in Hg.

1.2.3 Age Analysis

Null hypothesis: there is no statistically significant relationship between **Age (years)** and **Hg (mg/kg)**. Age will not significantly predict **Hg (mg/kg)**.

Hypothesis: **Age (years)** will significantly predict **Hg (mg/kg)**. As age increases, the concentration of **Hg (mg/kg)** will significantly increase.

Much of the methodology is the same as was done in the weight analysis.

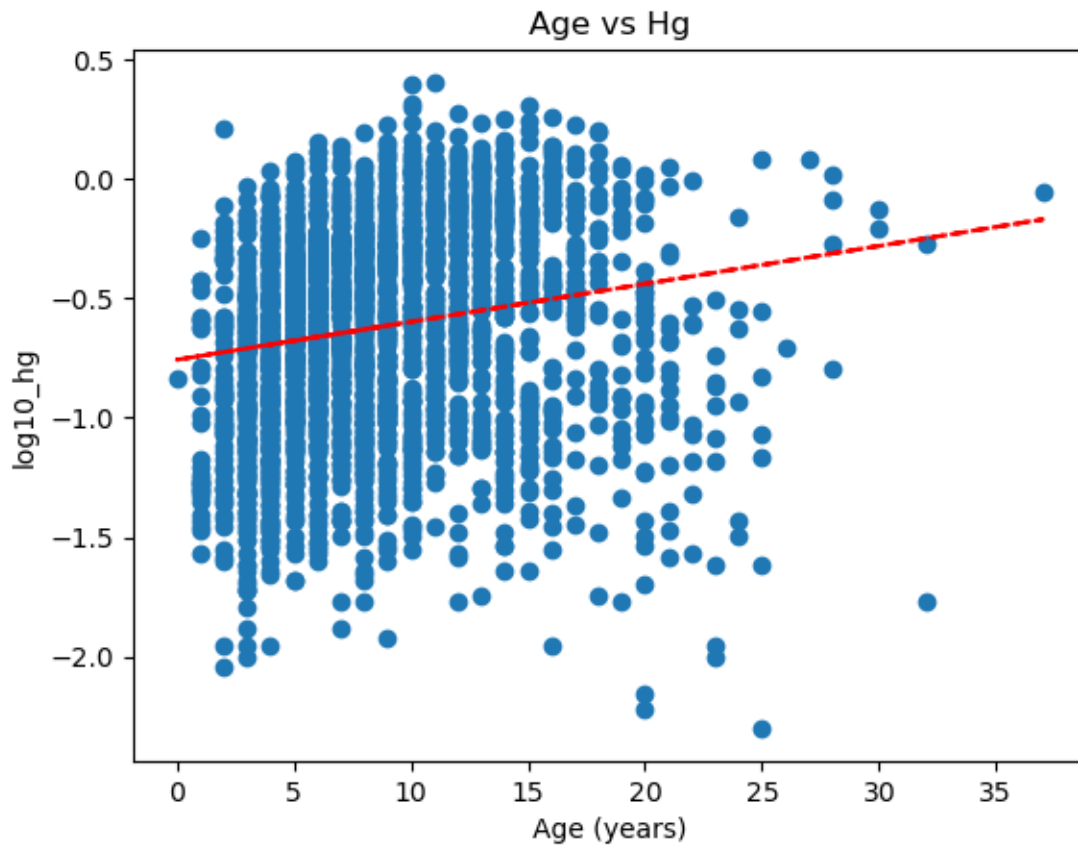
```
[36]: # Creating a new dataframe and dropping any NaN values
df_age = df[['Age (years)', 'log10_hg']].dropna()

[37]: # Creating a scatter plot with a trend line to investigate and visualize a
      ↪ potential linear relationship between Age (years) and Hg (mg/kg)
x = df_age['Age (years)']
y = df_age['log10_hg']

plt.scatter(x, y)

z = np.polyfit(x, y, deg=1)
p = np.poly1d(z)

plt.plot(x, p(x), 'r--')
plt.title('Age vs Hg')
plt.xlabel('Age (years)')
plt.ylabel('log10_hg')
plt.show()
```



There appears to be a linear relationship between Age (years) and Hg (mg/kg), a Pearson corre-

lation can be computed to quantify the relationship.

```
[38]: # Performing a Pearson correlation
corr, p_value = stats.pearsonr(df_age['Age (years)'], df_age['log10_hg'])
df_degree = len(df_age) - 2
print(f'The degrees of freedom is: {df_degree}')
print(f'The Pearsons correlation is: {corr:.3f}')
print(f'The p-value is: {p_value:.3f}')
```

The degrees of freedom is: 2975

The Pearsons correlation is: 0.177

The p-value is: 0.000

A Pearson correlation coefficient was computed to assess the linear relationship between Age (years) and Hg (mg/kg). There was a positive correlation between the two variables, $r(2975) = 0.177$, $p < 0.001$. Performing a linear regression could provide more information about the relationship between these variables.

```
[39]: # Performing a simple linear regression
y = df_age['log10_hg']
x = df_age['Age (years)']
x = sm.add_constant(x)
model = sm.OLS(y, x).fit()
model.summary()
```

[39]:

Dep. Variable:	log10_hg	R-squared:	0.031
Model:	OLS	Adj. R-squared:	0.031
Method:	Least Squares	F-statistic:	96.46
Date:	Tue, 05 Aug 2025	Prob (F-statistic):	1.99e-22
Time:	08:56:51	Log-Likelihood:	-1615.6
No. Observations:	2977	AIC:	3235.
Df Residuals:	2975	BIC:	3247.
Df Model:	1		
Covariance Type:	nonrobust		

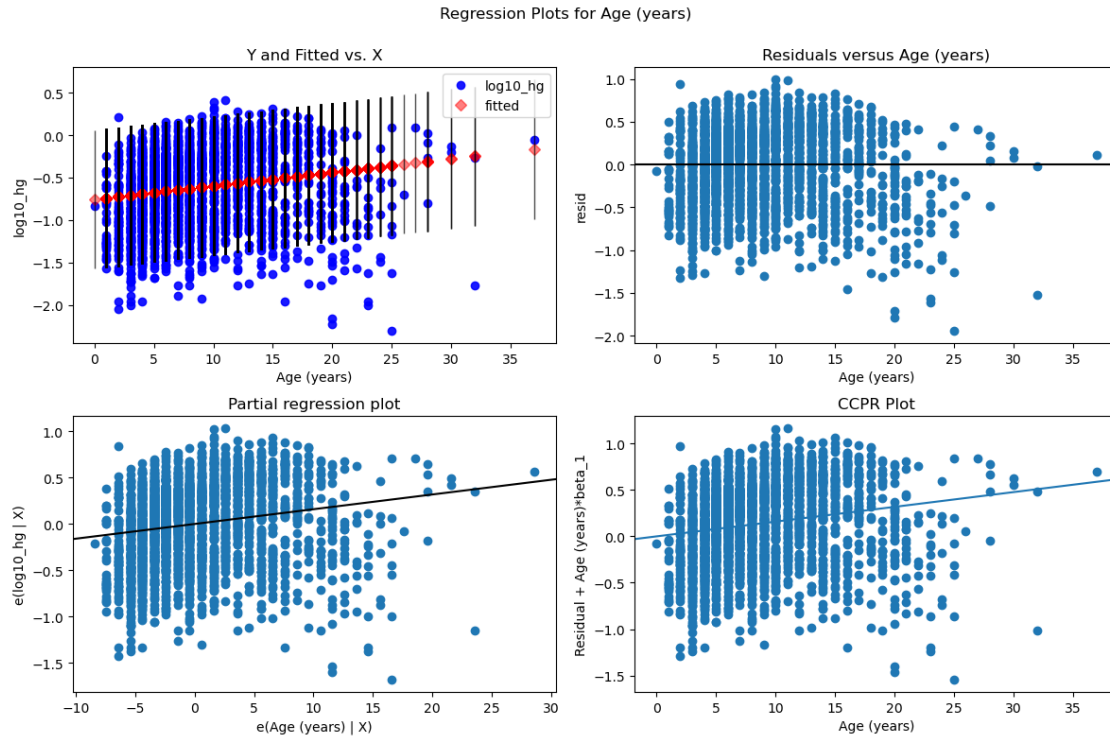
	coef	std err	t	P> t	[0.025	0.975]
const	-0.7564	0.016	-48.415	0.000	-0.787	-0.726
Age (years)	0.0159	0.002	9.822	0.000	0.013	0.019

Omnibus:	157.277	Durbin-Watson:	0.442
Prob(Omnibus):	0.000	Jarque-Bera (JB):	181.799
Skew:	-0.600	Prob(JB):	3.33e-40
Kurtosis:	3.156	Cond. No.	19.9

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[40]: # Creating residual plots to check linear regression assumptions
fig = plt.figure(figsize=(12, 8))
fig = sm.graphics.plot_regress_exog(model, "Age (years)", fig=fig)
```



The results from the linear regression are using the log10 transformed Hg (mg/kg) data. Reversing the transformation will give the constant needed to build the linear equation ($y = mx + b$).

```
[41]: # Undo the transformation of the constant for the linear regression model
b = np.exp(-0.7564)
print(f'The untransformed constant for the linear regression is: {b:.3f}')
```

The untransformed constant for the linear regression is: 0.469

A simple linear regression was used to test if Age (years) significantly predicted Hg (mg/kg). The fitted regression model was: **Hg (mg/kg) = 0.0159 x Age (years) + 0.469**. The overall regression was statistically significant; $R^2 = 0.031$, $F(1, 2975) = 96.46$, $p < 0.001$. The equation predicts a 0.0159mg increase in Hg concentration per increase in age by one year.

1.2.4 Sex Analysis

Null hypothesis: There are no significant differences in the means of Hg (mg/kg) between sexes (male and female).

Hypothesis: The null hypothesis cannot be rejected, there are no significant differences in the means of Hg (mg/kg) between sexes (male and female).

```
[42]: # Creating a new dataframe and dropping any NaN values
df_sex = df[['Sex', 'log10_hg', 'Hg (mg/kg)']].dropna()
```

```
[43]: # Stripping whitespace surrounding column values
df_sex['Sex'] = df_sex['Sex'].str.strip()
```

```
[44]: # Identifying all unique values in the column and their respective counts
df_sex['Sex'].value_counts()
```

```
[44]: Sex
      Female      2663
      Male       1876
      Unknown    1828
      Name: count, dtype: int64
```

This column also contains *Unknown* values, which will be dropped.

```
[45]: # Removing the rows with 'Unknown' values
df_sex = df_sex[~df_sex.isin(['Unknown']).any(axis=1)]
```

A two sample t-test can be performed to calculate if the means between male and female fish are significantly different. The data need to be separated into groups first.

```
[46]: # Separating the data into groups
group_male = df_sex[df_sex['Sex'] == 'Male']['log10_hg']
group_female = df_sex[df_sex['Sex'] == 'Female']['log10_hg']
```

```
[47]: # Performing the t-test
t_statistic, p_value = stats.ttest_ind(group_male, group_female)
df_degree = len(group_male) + len(group_female) - 2
print(f'The degrees of freedom are: {df_degree}')
print(f'The t-statistic is: {t_statistic:.3f}')
print(f'The p-value is: {p_value:.3f}')
```

```
The degrees of freedom are: 4537
The t-statistic is: -3.726
The p-value is: 0.000
```

A two sample t-test was performed to compare the means of Hg (mg/kg) between the female group and male group. There was a significant difference in Hg (mg/kg) between the female group ($M = 0.345$) and the male group ($M = 0.324$); $t(4537) = -3.726$, $p < 0.001$. The null hypothesis can be rejected.

```
[48]: # Calculating the mean values for each sex and sorting them from largest to
      ↪smallest
mean_values = df_sex.groupby('Sex')['Hg (mg/kg)'].mean().round(3).
      ↪sort_values(ascending=False)
print(mean_values)
```

```
Sex
      Female    0.345
      Male     0.324
      Name: Hg (mg/kg), dtype: float64
```

The analysis shows that female fish have a significantly higher concentration of Hg than male fish. Females have a mean of 0.345 mg/kg and males have a mean of 0.324 mg/kg.

1.2.5 Sex and Weight Analysis

The previous results show that the concentration of Hg (mg/kg) is significantly different between male and female fish. To further investigate this, an analysis will be performed to determine if the mean of Weight (g) is significantly different between Sex (male and female).

Null hypothesis: There are no significant differences in the mean weights between male and female fish.

Hypothesis: There is a significant difference between the mean weights between male and female fish. Female fish have a significantly larger mean weight than male fish.

```
[49]: # Stripping leading and trailing white space from the values in the 'Sex' column
df['Sex'] = df['Sex'].str.strip()
```

```
[50]: # Creating a new dataframe with only the relevant data
df_sex_weight = df[['Sex', 'Weight (g)']].dropna()
```

```
[51]: # Removing the 'Unknown' values from the dataframe
df_sex_weight = df_sex_weight[~df_sex_weight.isin(['Unknown']).any(axis=1)]
```

```
[52]: # Separating the data into groups for the t-test
male_weights = df_sex_weight[df_sex_weight['Sex'] == 'Male']['Weight (g)']
female_weights = df_sex_weight[df_sex_weight['Sex'] == 'Female']['Weight (g)']
```

```
[53]: # Performing the t-test
t_statistic, p_value = stats.ttest_ind(male_weights, female_weights)
df_degree = len(male_weights) + len(female_weights) - 2
print(f'The degrees of freedom are: {df_degree}')
print(f'The t-statistic is: {t_statistic:.3f}')
print(f'The p-value is: {p_value:.3f}')
```

The degrees of freedom are: 4535

The t-statistic is: -13.948

The p-value is: 0.000

```
[54]: # Calculating the mean values of each sex and ordering them from largest to
      ↪smallest
mean_values = df_sex_weight.groupby('Sex')['Weight (g)'].mean().round(1).
      ↪sort_values(ascending=False)
print(mean_values)
```

Sex

Female 1718.6

Male 1258.0

Name: Weight (g), dtype: float64

A two sample t-test was performed to compare the means of **Weight (g)** between the female fish and male fish. There was a significant difference between the female group ($M = 1718.6g$) and the male group ($M = 1258.0g$); $t(4535) = -13.948$, $p < 0.001$. The null hypothesis can be rejected.

1.2.6 Sex and Age Analysis

To further investigate the possible variables affecting the larger concentration of Hg (mg/kg) in female fish, the relationship between **Sex** and **Age (years)** will also be analyzed.

Null hypothesis: There is no significant difference between the mean Hg (mg/kg) concentration of female and male fish.

Hypothesis: There is a significant difference between the mean Hg (mg/kg) concentration of female and male fish.

```
[55]: # Creating a new dataframe with only the Sex and Age (years) columns and
      ↪ dropping any NaN values
df_sex_age = df[['Sex', 'Age (years)']].dropna()

# Removing the 'Unknown' variables from the Sex column
df_sex_age = df_sex_age[~df_sex_age.isin(['Unknown']).any(axis=1)]
```

```
[56]: # Separating the data into groups for the t-test
male_ages = df_sex_age[df_sex_age['Sex'] == 'Male']['Age (years)']
female_ages = df_sex_age[df_sex_age['Sex'] == 'Female']['Age (years)']
```

```
[57]: # Performing the t-test
t_statistic, p_value = stats.ttest_ind(male_ages, female_ages)
df_degree = len(male_ages) + len(female_ages) - 2
print(f'The degrees of freedom are: {df_degree}')
print(f'The t-statistic is: {t_statistic:.3f}')
print(f'The p-value is: {p_value:.3f}')
```

The degrees of freedom are: 2967

The t-statistic is: 5.417

The p-value is: 0.000

```
[58]: # Calculating the mean values and ordering them by largest to smallest
mean_values = df_sex_age.groupby('Sex')['Age (years)'].mean().round(2).
      ↪ sort_values(ascending=False)
print(mean_values)
```

Sex

Male 8.97

Female 8.02

Name: Age (years), dtype: float64

A two sample t-test was performed to compare the means of **Age (years)** between female fish and male fish. There was a significant difference between female ($M = 8.02$ years) and male ($M = 8.97$ years) fish; $t(2967) = 5.417$, $p < 0.001$. The null hypothesis can be rejected.

Male fish were found to be significantly older than female fish.

1.2.7 Fork Length Analysis

Null hypothesis: There is no significant relationship between Fork Length (mm) and Hg (mg/kg) concentration. Fork Length (mm) will not significantly predict Hg (mg/kg).

Hypothesis: There is a significant relationship between Fork Length (mm) and Hg (mg/kg) concentration. Fork Length (mm) will significantly predict Hg (mg/kg). As Fork Length (mm) increases the Hg (mg/kg) concentration will significantly increase.

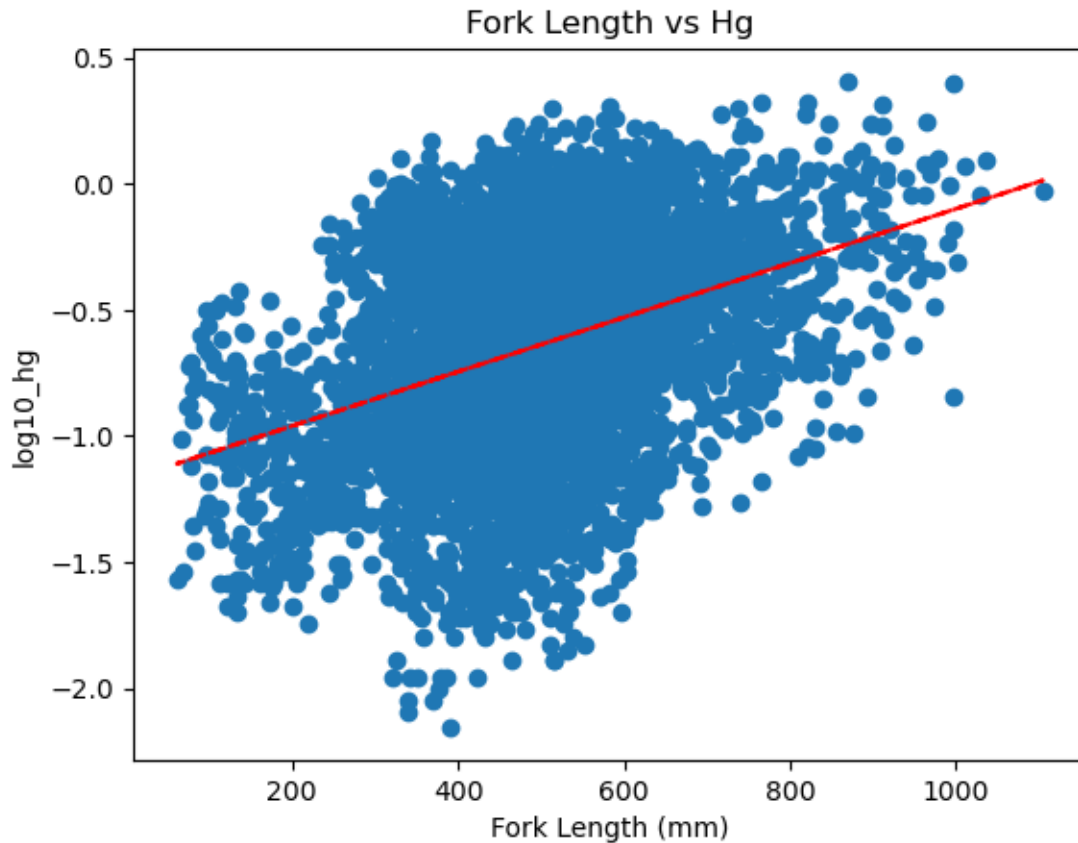
```
[59]: # Creating new dataframe with only relevant data and dropping any NaN values
df_fork = df[['Fork Length (mm)', 'log10_hg', 'Hg (mg/kg)']].dropna()

[60]: # Creating a scatter plot with a trend line to visualize the potential linear
      ↪ relationship between Fork Length (mm) and Hg (mg/kg)
x = df_fork['Fork Length (mm)']
y = df_fork['log10_hg']

plt.scatter(x, y)

z = np.polyfit(x, y, deg=1)
p = np.poly1d(z)

plt.plot(x, p(x), 'r--')
plt.title('Fork Length vs Hg')
plt.xlabel('Fork Length (mm)')
plt.ylabel('log10_hg')
plt.show()
```



```
[61]: # Performing a Pearson correlation
corr, p_value = stats.pearsonr(df_fork['Fork Length (mm)'], df_fork['log10_hg'])
df_degree = len(df_age) - 2
print(f'The degrees of freedom is: {df_degree}')
print(f'The Pearsons correlation is: {corr:.3f}')
print(f'The p-value is: {p_value:.3f}')
```

The degrees of freedom is: 2975
The Pearsons correlation is: 0.375
The p-value is: 0.000

A Pearson correlation coefficient was computed to assess the linear relationship between **Fork Length (mm)** and **Hg (mg/kg)**. There was a positive correlation between the two variables, $r(2975) = 0.375$, $p < 0.001$. Performing a linear regression could provide more information about the relationship between these variables.

```
[62]: # Performing a simple linear regression
y = df_fork['log10_hg']
x = df_fork['Fork Length (mm)']
x = sm.add_constant(x)
model = sm.OLS(y, x).fit()
```

```
model.summary()
```

[62]:

Dep. Variable:	log10_hg	R-squared:	0.141
Model:	OLS	Adj. R-squared:	0.140
Method:	Least Squares	F-statistic:	863.5
Date:	Tue, 05 Aug 2025	Prob (F-statistic):	6.35e-176
Time:	08:56:51	Log-Likelihood:	-2548.8
No. Observations:	5284	AIC:	5102.
Df Residuals:	5282	BIC:	5115.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	-1.1755	0.019	-62.443	0.000	-1.212	-1.139
Fork Length (mm)	0.0011	3.66e-05	29.385	0.000	0.001	0.001

Omnibus:	108.074	Durbin-Watson:	0.398
Prob(Omnibus):	0.000	Jarque-Bera (JB):	110.248
Skew:	-0.336	Prob(JB):	1.15e-24
Kurtosis:	2.776	Cond. No.	1.79e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

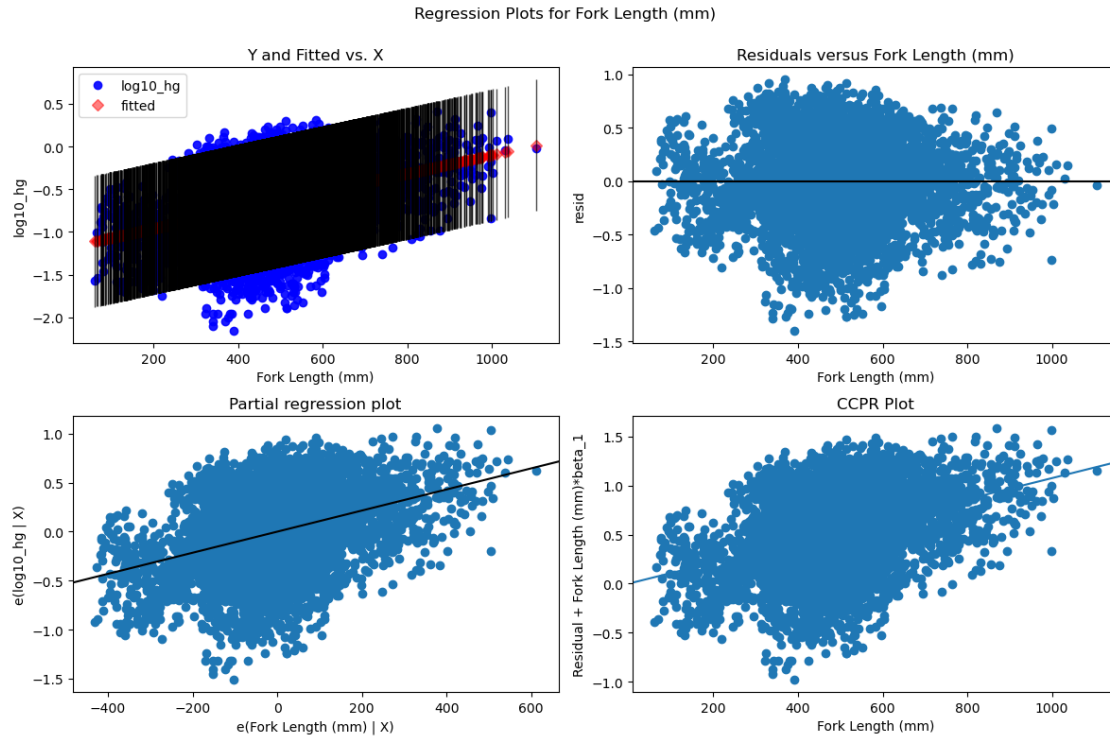
[2] The condition number is large, 1.79e+03. This might indicate that there are strong multicollinearity or other numerical problems.

The results from the linear regression are using the log10 transformed Hg (mg/kg) data. Reversing the transformation will give the constant needed to build the linear equation ($y = mx + b$).

```
[63]: # Reverse the transformation of the constant for the linear regression model
b = np.exp(-1.1755)
print(f'The untransformed constant for the linear regression is: {b:.3f}')
```

The untransformed constant for the linear regression is: 0.309

```
[64]: # Creating residual plots to check linear regression assumptions
fig = plt.figure(figsize=(12, 8))
fig = sm.graphics.plot_regress_exog(model, "Fork Length (mm)", fig=fig)
```



A simple linear regression was used to test if Fork Length (mm) significantly predicted Hg (mg/kg). The fitted regression model was: $\text{Hg (mg/kg)} = 0.0011 \times \text{Fork Length (mm)} + 0.309$. The overall regression was statistically significant; $R^2 = 0.141$, $F(1, 5282) = 863.5$, $p < 0.001$.

1.2.8 Total Length Analysis

Null hypothesis: There is no significant relationship between Total Length (mm) and Hg (mg/kg) concentration. Total Length (mm) will not significantly predict Hg (mg/kg).

Hypothesis: There is a significant relationship between Total Length (mm) and Hg (mg/kg) concentration; Total Length (mm) will significantly predict Hg (mg/kg). As total length increases the Hg (mg/kg) concentration will significantly increase.

```
[65]: # Creating new dataframe with only relevant data and dropping any NaN values
df_total = df[['Total Length (mm)', 'log10_hg', 'Hg (mg/kg)']].dropna()
```

```
[66]: # Creating a scatter plot with a trend line to visualize the potential linear
      ↪ relationship between Total Length (mm) and Hg (mg/kg)
x = df_total['Total Length (mm)']
y = df_total['log10_hg']

plt.scatter(x, y)

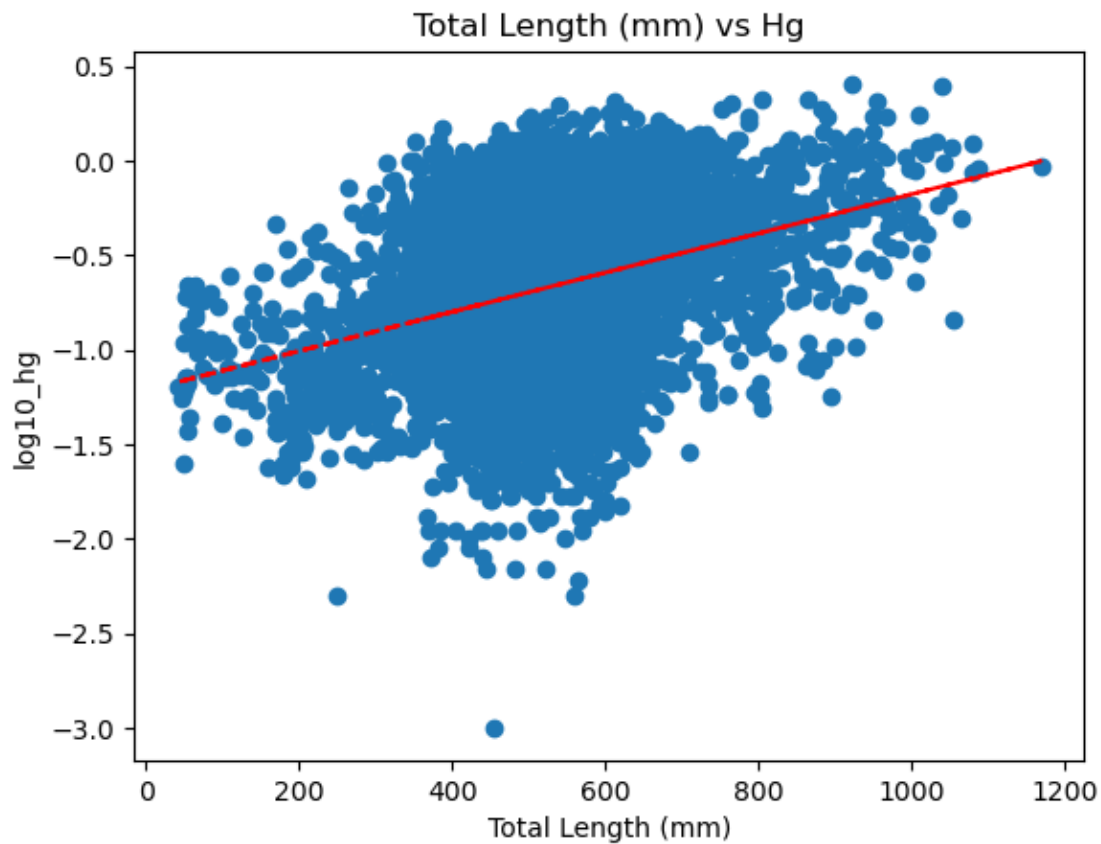
z = np.polyfit(x, y, deg=1)
```

```

p = np.poly1d(z)

plt.plot(x, p(x), 'r--')
plt.title('Total Length (mm) vs Hg')
plt.xlabel('Total Length (mm)')
plt.ylabel('log10_hg')
plt.show()

```



```

[67]: # Performing a Pearson correlation
corr, p_value = stats.pearsonr(df_total['Total Length (mm)'],
    ↪df_total['log10_hg'])
df_degree = len(df_total) - 2
print(f'The degrees of freedom is: {df_degree}')
print(f'The Pearsons correlation is: {corr:.3f}')
print(f'The p-value is: {p_value:.3f}')

```

The degrees of freedom is: 5171
 The Pearsons correlation is: 0.356
 The p-value is: 0.000

A Pearson correlation coefficient was computed to assess the linear relationship between Total

Length (mm) and Hg (mg/kg). There was a positive correlation between the two variables, $r(5171) = 0.356$, $p < 0.001$. Performing a linear regression could provide more information about the relationship between these variables.

```
[68]: # Performing a Pearson correlation
y = df_total['log10_hg']
x = df_total['Total Length (mm)']
x = sm.add_constant(x)
model = sm.OLS(y, x).fit()
model.summary()
```

[68]:

Dep. Variable:	log10_hg	R-squared:	0.126
Model:	OLS	Adj. R-squared:	0.126
Method:	Least Squares	F-statistic:	748.1
Date:	Tue, 05 Aug 2025	Prob (F-statistic):	5.90e-154
Time:	08:56:52	Log-Likelihood:	-2683.5
No. Observations:	5173	AIC:	5371.
Df Residuals:	5171	BIC:	5384.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	-1.2143	0.021	-58.262	0.000	-1.255	-1.173
Total Length (mm)	0.0010	3.79e-05	27.352	0.000	0.001	0.001

Omnibus:	142.048	Durbin-Watson:	0.398
Prob(Omnibus):	0.000	Jarque-Bera (JB):	153.393
Skew:	-0.420	Prob(JB):	4.91e-34
Kurtosis:	3.068	Cond. No.	2.02e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

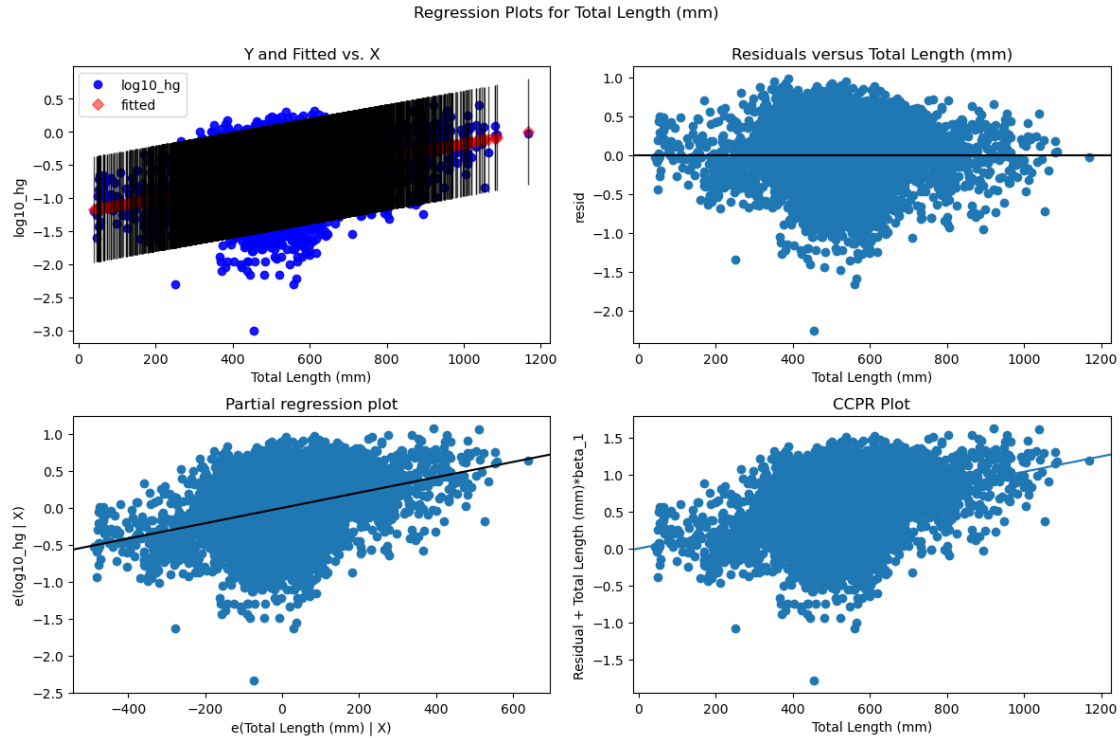
[2] The condition number is large, 2.02e+03. This might indicate that there are strong multicollinearity or other numerical problems.

The results from the linear regression are using the log10 transformed Hg (mg/kg) data. Reversing the transformation will give the constant needed to build the linear equation ($y = mx + b$).

```
[69]: # Reverse the transformation of the constant for the linear regression model
b = np.exp(-1.2143)
print(f'The untransformed constant for the linear regression is: {b:.3f}')
```

The untransformed constant for the linear regression is: 0.297

```
[70]: # Creating residual plots to check linear regression assumptions
fig = plt.figure(figsize=(12, 8))
fig = sm.graphics.plot_regress_exog(model, "Total Length (mm)", fig=fig)
```



Simple linear regression was used to test if **Total Length (mm)** significantly predicted **Hg (mg/kg)**. The fitted regression model was: **Hg (mg/kg) = 0.0010 x Total Length (mm) + 0.297**. The overall regression was statistically significant; $R^2 = 0.126$, $F(1, 5171) = 748.1$, $p < 0.001$.

1.2.9 Waterbody Type Analysis

Null hypothesis: There are no significant differences in the mean **Hg (mg/kg)** concentration of fish between any pair of waterbody types.

Hypothesis: There are significant differences in the mean **Hg (mg/kg)** concentration of fish in at least one pair of waterbody types.

```
[71]: # Creating new dataframe and dropping any NaN values
df_type = df[['Waterbody Type', 'log10_hg', 'Hg (mg/kg)']].dropna()

[72]: # Stripping whitespace surrounding column values
df_type['Waterbody Type'] = df_type['Waterbody Type'].str.strip()

[73]: # Identifying all unique values in the column and their respective counts
df_type['Waterbody Type'].value_counts()
```

```
[73]: Waterbody Type
      Lake      4419
      River     1230
      Reservoir    685
```

```
Stormwater Pond      18
Canal                 15
Name: count, dtype: int64
```

```
[74]: # Separating the data into groups
waterbody_types = ['Lake', 'River', 'Reservoir', 'Stormwater Pond', 'Canal']

groups = {}
for waterbody in waterbody_types:
    groups[waterbody] = df_type[df_type['Waterbody Type'] ==
    ↪waterbody]['log10_hg'].dropna()

# Number of groups
num_groups = len(groups)

# The total number of observations
num_observations = sum([len(group) for group in groups.values()])

# Computing degrees of freedom
df_between_groups = num_groups - 1
df_within_groups = num_observations - num_groups
print(f'Between-groups Degrees of Freedom: {df_between_groups}')
print(f'Within-groups Degrees of Freedom: {df_within_groups}')

# Performing the one-way ANOVA using values from the dictionary
F_statistic, p_value = stats.f_oneway(*groups.values())
print(f'The F-statistic is: {F_statistic:.3f}')
print(f'The p-value is: {p_value:.3f}')
```

```
Between-groups Degrees of Freedom: 4
Within-groups Degrees of Freedom: 6362
The F-statistic is: 70.768
The p-value is: 0.000
```

A one-way ANOVA was performed to compare the effect of Waterbody Type on Hg (mg/kg). A one-way ANOVA revealed that there was a statistically significant difference in Hg (mg/kg) between at least two groups; $F(4, 6362) = 70.768$, $p < 0.001$. A Tukey HSD test can be used to identify which variables have a significant difference between their means.

```
[75]: # Create a new DataFrame with structure suitable for the Tukey HSD test
dataframes = []
groups = ['Lake', 'River', 'Reservoir', 'Stormwater Pond', 'Canal']
for group in groups:
    df_type_group = df_type[df_type['Waterbody Type'] == group][['log10_hg']].
    ↪copy()
    df_type_group['group'] = group
    dataframes.append(df_type_group)
tukey_df_type = pd.concat(dataframes)
```

```
# Drop NaN values to prevent errors
tukey_df_type = tukey_df_type.dropna()

# Apply the Tukey HSD test
results = pairwise_tukeyhsd(endog=tukey_df_type['log10_hg'],
    ↳groups=tukey_df_type['group'], alpha=0.05)
print(results)
```

Multiple Comparison of Means - Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
Canal	Lake	0.3094	0.0374	0.0113	0.6075	True
Canal	Reservoir	0.5622	0.0	0.2614	0.8631	True
Canal	River	0.2671	0.1064	-0.0323	0.5665	False
Canal	Stormwater Pond	-0.2125	0.6024	-0.6155	0.1905	False
Lake	Reservoir	0.2528	0.0	0.2055	0.3001	True
Lake	River	-0.0423	0.0164	-0.0794	-0.0051	True
Lake	Stormwater Pond	-0.5219	0.0	-0.7941	-0.2497	True
Reservoir	River	-0.2951	0.0	-0.35	-0.2401	True
Reservoir	Stormwater Pond	-0.7747	0.0	-1.0499	-0.4995	True
River	Stormwater Pond	-0.4796	0.0	-0.7533	-0.206	True

The Tukey HSD Test showed there was a significant difference between the means of Hg (mg/kg) in following groups: Canal & Lake, Canal & Reservoir, Lake & Reservoir, Lake & River, Lake & Stormwater Pond, Reservoir & River, Reservoir & Stormwater Pond, and River & Stormwater Pond.

However, the test showed no significant difference between the following pairs of waterbody types: Canal & River and Canal & Stormwater Pond.

```
[76]: # Calculate the mean Hg (mg/kg) value for each water body type (group) and sort
    ↳the means largest to smallest
mean_values = df_type.groupby('Waterbody Type')['Hg (mg/kg)'].mean().round(3).
    ↳sort_values(ascending=False)
print(mean_values)
```

```
Waterbody Type
Reservoir      0.480
Lake           0.311
River          0.268
Canal          0.120
Stormwater Pond 0.062
Name: Hg (mg/kg), dtype: float64
```

The analysis shows that type Reservoir had the highest concentration of Hg (mg/kg) at 0.480 mg/kg and type Stormwater Pond had the lowest at 0.062 mg/kg.

```
[77]: # Plotting a boxplot to visualize the distribution of data for all groups
waterbody_means = px.box(df_type, x='Waterbody Type', y='log10_hg')
waterbody_means.show()
```

1.2.10 Waterbody Name Analysis

This is an exploratory analysis and no statistical tests will be performed.

```
[78]: # Creating new dataframe with only relevant data and dropping any NaN values
df_name = df[['Waterbody Name', 'Hg (mg/kg)']].dropna()
```

```
[79]: # Stripping whitespace surrounding column values
df_name['Waterbody Name'] = df_name['Waterbody Name'].str.strip()
```

```
[80]: # Identifying all unique values in the column and their respective counts
df_name['Waterbody Name'].value_counts()
```

```
[80]: Waterbody Name
Willow Creek                459
Moose Lake                  196
Twin Valley Reservoir       142
Shiningbank Lake            130
Pigeon Lake                  128
...
North Saskatchewan River (within Edmonton)    5
Loon River                                     4
Hay River                                      4
North Saskatchewan River (between Edmonton and Devon)  2
Lac La Biche                                  1
Name: count, Length: 142, dtype: int64
```

```
[81]: # Calculating the means of Hg (mg/kg) for each Waterbody Name and sorting them
↳from largest to smallest
sorted_grouped = df_name.groupby('Waterbody Name')['Hg (mg/kg)'].mean().
↳round(3).sort_values(ascending=False)
print(sorted_grouped)
```

```
Waterbody Name
Rolling Hills Lake (Rolling Hills Reservoir)    1.088
Narrow Lake                                     0.897
Clear Lake (Barnes Lake)                       0.887
Ironwood Lake                                  0.861
Burnstick Lake                                 0.818
...
Richardson Lake                                0.068
Athabasca River (between Hinton and Whitecourt) 0.067
County Sportsplex Pond                         0.062
Snipe Lake                                     0.049
```

McLeod Lake (Carson Lake) 0.049
Name: Hg (mg/kg), Length: 142, dtype: float64

```
[82]: # Creating a new dataframe with just the waterbody name and the mean Hg (mg/kg)
df_avoid = sorted_grouped.to_frame().reset_index()
print(df_avoid)
```

	Waterbody Name	Hg (mg/kg)
0	Rolling Hills Lake (Rolling Hills Reservoir)	1.088
1	Narrow Lake	0.897
2	Clear Lake (Barnes Lake)	0.887
3	Ironwood Lake	0.861
4	Burnstick Lake	0.818
..	...	...
137	Richardson Lake	0.068
138	Athabasca River (between Hinton and Whitecourt)	0.067
139	County Sportsplex Pond	0.062
140	Snipe Lake	0.049
141	McLeod Lake (Carson Lake)	0.049

[142 rows x 2 columns]

The analysis shows the highest Hg (mg/kg) was in Rolling Hills Lake (Rolling Hills Reservoir) at 1.09 mg/kg and the lowest Hg (mg/kg) was in McLeod Lake (Carson Lake) at 0.049 mg/kg.

According to a 2019 Government of Alberta Report, Hg (mg/kg) concentrations above 0.5 mg/kg are given **Avoid Consumption** advice. The list of waterbodies that should be avoided can be computed using a python function.

```
[83]: # Creating the function
def avoid_consumption(val):
    if val >= 0.5:
        return True
    else:
        return False

# Applying the function to the Hg (mg/kg) column and creating a new column
# called 'check'
df_avoid['check'] = df_avoid['Hg (mg/kg)'].apply(avoid_consumption)

# Updating the dataframe to contain only the check values that equal True
df_avoid = df_avoid[df_avoid['check'] == True][['Waterbody Name', 'Hg (mg/kg)']]

# Updating the dataframe to only contain Waterbody Name and Hg (mg/kg) which
# has been filtered by the check - true filter
df_avoid = df_avoid[['Waterbody Name', 'Hg (mg/kg)']]

# Printing the dataframe
```

```
print('The waterbodies with an average Hg (mg/kg) concentration above_
↳recommended levels in its fish are:', df_avoid)
```

The waterbodies with an average Hg (mg/kg) concentration above recommended levels in its fish are:

	Waterbody Name	Hg (mg/kg)
0	Rolling Hills Lake (Rolling Hills Reservoir)	1.088
1	Narrow Lake	0.897
2	Clear Lake (Barnes Lake)	0.887
3	Ironwood Lake	0.861
4	Burnstick Lake	0.818
5	Rolling Hills Lake	0.777
6	Pitchimi Lake	0.748
7	Garner Lake	0.725
8	Red Deer River (Hwy 585 Crossing - Tolman Bridge)	0.686
9	Amisk Lake	0.684
10	McGregor Lake	0.674
11	Hilda Lake	0.658
12	Pinehurst Lake	0.655
13	Whitefish Lake	0.647
14	Iosegun Lake	0.637
15	Hay River	0.606
16	Touchwood Lake	0.601
17	Soar's Lake	0.598
18	Pine Coulee Reservoir	0.596
19	Crane Lake (Moore Lake)	0.564
20	Sherburne Lake	0.563
21	40 Mile Reservoir	0.550
22	Kinnaird Lake	0.545
23	Cowoki Lake (Cowoki Reservoir)	0.542
24	Red Deer River (between Bleriot Ferry and down...	0.537
25	South Saskatchewan River (at Medicine Hat)	0.523
26	Kehiwin Lake (Kehewin Lake)	0.508
27	Crawling Valley Reservoir (Barkenhouse Lake)	0.500

```
[84]: # Creating a dataframe with a list of the waterbody names
df_avoid_names = df_avoid['Waterbody Name'].unique()

# Converting the dataframe to a string to make it more readable when printed
df_avoid_names_str = ', '.join(df_avoid_names)

print('There are', df_avoid['Waterbody Name'].count(), 'waterbodies you should_
↳avoid fishing in. They are:', df_avoid_names_str)
```

There are 28 waterbodies you should avoid fishing in. They are: Rolling Hills Lake (Rolling Hills Reservoir), Narrow Lake, Clear Lake (Barnes Lake), Ironwood Lake, Burnstick Lake, Rolling Hills Lake, Pitchimi Lake, Garner Lake, Red Deer River (Hwy 585 Crossing - Tolman Bridge), Amisk Lake, McGregor Lake, Hilda Lake,

Pinehurst Lake, Whitefish Lake, Iosegun Lake, Hay River, Touchwood Lake, Soar's Lake, Pine Coulee Reservoir, Crane Lake (Moore Lake), Sherburne Lake, 40 Mile Reservoir, Kinnaird Lake, Cowoki Lake (Cowoki Reservoir), Red Deer River (between Bleriot Ferry and downstream of Drumheller), South Saskatchewan River (at Medicine Hat), Kehiwin Lake (Kehewin Lake), Crawling Valley Reservoir (Barkenhouse Lake)

1.2.11 Common Name Analysis

This is an exploratory analysis and no statistical tests will be performed.

```
[85]: # Creating new dataframe with only relevant data and dropping any NaN values
df_common = df[['Common Name', 'log10_hg', 'Hg (mg/kg)']].dropna()
```

```
[86]: # Stripping whitespace surrounding column values
df_common['Common Name'] = df_common['Common Name'].str.strip()
```

```
[87]: # Identifying all unique values in the column and their respective counts
df_common['Common Name'].value_counts()
```

```
[87]: Common Name
Walleye                2042
Northern Pike          2014
Lake Whitefish         932
White Sucker           394
Longnose Sucker        154
Mountain Whitefish     151
Lake Trout             104
Yellow Perch           92
Burbot                 89
Rainbow Trout          79
Brook Trout            65
Goldeye                60
Cisco                  54
Sauger                 43
Shorthead Redhorse     25
Mooneye                18
Lake Chub (Cisco)      16
Brown Trout            13
Longnose Dace          10
Trout Perch            9
Cutthroat Trout        2
Flathead Chub          1
Name: count, dtype: int64
```

```
[88]:
```

```
# Calculating the means of Hg (mg/kg) for each group and sorting them from
↳largest to smallest
sorted_grouped = df_common.groupby('Common Name')['Hg (mg/kg)'].mean().round(3).
↳sort_values(ascending=False)
print(sorted_grouped)
```

```
Common Name
Sauger          0.747
Goldeye         0.474
Walleye         0.462
Shorthead Redhorse 0.442
Northern Pike   0.361
Lake Trout      0.313
Mooneye         0.302
Burbot          0.301
White Sucker    0.198
Flathead Chub   0.174
Lake Chub (Cisco) 0.153
Longnose Dace   0.141
Brook Trout     0.140
Longnose Sucker 0.135
Brown Trout     0.096
Cisco           0.091
Lake Whitefish  0.089
Yellow Perch    0.089
Mountain Whitefish 0.078
Trout Perch     0.076
Rainbow Trout   0.074
Cutthroat Trout 0.044
Name: Hg (mg/kg), dtype: float64
```

A function can be created that will sort the names of fish based on consumption ranges defined by the 2019 Government of Alberta report.

```
[89]: # Creating a new dataframe that contains the common names and their mean Hg (mg/
↳kg) concentration as columns
df_sorted_grouped = sorted_grouped.to_frame().reset_index()
```

```
[90]: # Creating a function that will label a value based on the conditions in the
↳function and insert them into a new column
def range_based_rules(val):
    if val < 0.2:
        return 'No Consumption Advice'
    elif 0.2 <= val < 0.5:
        return 'Consumption Limit'
    else:
        return 'Avoid Consumption'
```

```
df_sorted_grouped['Consumption Advice'] = df_sorted_grouped['Hg (mg/kg)'].
↳apply(range_based_rules)
```

```
[91]: # Printing the new dataframe with the new column
print(df_sorted_grouped)
```

	Common Name	Hg (mg/kg)	Consumption Advice
0	Sauger	0.747	Avoid Consumption
1	Goldeye	0.474	Consumption Limit
2	Walleye	0.462	Consumption Limit
3	Shorthead Redhorse	0.442	Consumption Limit
4	Northern Pike	0.361	Consumption Limit
5	Lake Trout	0.313	Consumption Limit
6	Mooneye	0.302	Consumption Limit
7	Burbot	0.301	Consumption Limit
8	White Sucker	0.198	No Consumption Advice
9	Flathead Chub	0.174	No Consumption Advice
10	Lake Chub (Cisco)	0.153	No Consumption Advice
11	Longnose Dace	0.141	No Consumption Advice
12	Brook Trout	0.140	No Consumption Advice
13	Longnose Sucker	0.135	No Consumption Advice
14	Brown Trout	0.096	No Consumption Advice
15	Cisco	0.091	No Consumption Advice
16	Lake Whitefish	0.089	No Consumption Advice
17	Yellow Perch	0.089	No Consumption Advice
18	Mountain Whitefish	0.078	No Consumption Advice
19	Trout Perch	0.076	No Consumption Advice
20	Rainbow Trout	0.074	No Consumption Advice
21	Cutthroat Trout	0.044	No Consumption Advice

This information can be put together in the following way:

```
[92]: avoid_consumption_count = df_sorted_grouped[df_sorted_grouped['Consumption_
↳Advice'] == 'Avoid Consumption']['Consumption Advice'].count()
consumption_limit_count = df_sorted_grouped[df_sorted_grouped['Consumption_
↳Advice'] == 'Consumption Limit']['Consumption Advice'].count()
no_consumption_advice_count = df_sorted_grouped[df_sorted_grouped['Consumption_
↳Advice'] == 'No Consumption Advice']['Consumption Advice'].count()

avoid_consumption_names = df_sorted_grouped[df_sorted_grouped['Consumption_
↳Advice'] == 'Avoid Consumption']['Common Name'].unique()
consumption_limit_names = df_sorted_grouped[df_sorted_grouped['Consumption_
↳Advice'] == 'Consumption Limit']['Common Name'].unique()
no_consumption_advice_names = df_sorted_grouped[df_sorted_grouped['Consumption_
↳Advice'] == 'No Consumption Advice']['Common Name'].unique()

avoid_consumption_names_str = ', '.join(avoid_consumption_names)
consumption_limit_names_str = ', '.join(consumption_limit_names)
```

```

no_consumption_advice_names_str = ', '.join(no_consumption_advice_names)

print('There are', avoid_consumption_count, 'fish you should avoid consuming.␣
↳They are:', avoid_consumption_names_str)
print('')
print('There are', consumption_limit_count, 'fish you should limit consuming.␣
↳They are:', consumption_limit_names_str)
print('')
print('There are', no_consumption_advice_count, 'fish with no limit on␣
↳consuming. They are:', no_consumption_advice_names_str)

```

There are 1 fish you should avoid consuming. They are: Sauger

There are 7 fish you should limit consuming. They are: Goldeye, Walleye, Shorthead Redhorse, Northern Pike, Lake Trout, Mooneye, Burbot

There are 14 fish with no limit on consuming. They are: White Sucker, Flathead Chub, Lake Chub (Cisco), Longnose Dace, Brook Trout, Longnose Sucker, Brown Trout, Cisco, Lake Whitefish, Yellow Perch, Mountain Whitefish, Trout Perch, Rainbow Trout, Cutthroat Trout

These guidelines are taken from the report **Government of Alberta. (2019). *Fish Consumption Guidance: Mercury in Fish***

1.2.12 Results and Discussion

In general, the variables that influence the size and amount of tissue that a fish has were shown to significantly affect the concentration of Hg found in those fish; the larger the size, the higher concentration of Hg. These variables include **Weight (g)**, **Fork Length (mm)**, **Total Length (mm)**.

Similarly, the variables that are associated with time were also shown to significantly affect the concentration of Hg in those fish. Those variables are **Age (years)** and **Maturity**. As age increases, the more accumulation of Hg will occur. Likewise, fish that are further along in their development will have a larger concentration of Hg than those that are not.

The **Sex** variable was significant in determining the concentration of Hg, which was not the expected result. This could be explained by **sexual dimorphism**, so further analysis was performed to determine if the means of **Weight (g)** were significantly different between female and male sexes. They were found to be significantly different, with female fish weighing significantly more than male fish.

The relationship between **Sex** and **Age (years)** was also analyzed to help describe this observation. Males were found to be significantly older than females, on average. So sexual dimorphism could explain the higher concentration of Hg in females over males, but the extent of its influence could be impaired by the age of the fish. The interaction between **Weight (g)** and **Age (g)** and its effect on **Hg (mg/kg)** is not known. More research would need to be done, but the data seem to support this overall analysis.

Analyzing the **Watertype** variable gave results that generally aligned with the hypothesis tested. Waterbody types that have stagnated water (Reservoir and Lake) had significantly higher mean

concentrations of Hg (mg/kg) in their fish than those that have moving water (River, Canal, and Stormwater Pond).

There are **28** waterbodies you should avoid fishing in. These waterbodies have fish with an average of at least 0.5 mg/kg of Hg, which are given consumption advice of “avoid consumption” by the Government of Alberta.

A total of **22** species of fish are sampled in this dataset. There is one species of fish that is given the consumption advice of “avoid consumption”. There are 7 species that are given the advice of “consumption limit”. Finally, there are 14 species that are given no consumption advice.

1.2.13 References

Alberta Health Environmental Public Health Science Team. (n.d.). Chemical Monitoring in Local Foods: Mercury in Fish—Open Government. Retrieved August 14, 2024, from <https://open.alberta.ca/opendata/chemical-monitoring-in-local-foods-mercury-in-fish>

Government of Alberta. (2019). *Fish Consumption Guidance: Mercury in Fish*. Environmental Public Health Science Unit, Health Protection Branch, Public Health and Compliance Division, Alberta Health. Edmonton, Alberta.